

BLOQUE III - TEMA 2

DESARROLLO DE SISTEMAS

Lenguajes de programación.
Representación de tipos de
datos. Operadores. Instrucciones
condicionales. Bucles y recursividad.
Procedimientos, funciones y
parámetros. Vectores y registros.
Estructura de un programa.



ÍNDICE

1. LENGUAJES DE PROGRAMACIÓN	7
1.1. Tipos de lenguajes	7
1.1.1. Lenguajes de bajo nivel	7
1.1.2. Lenguajes de alto nivel	7
1.2. Lenguajes compilados e interpretados	7
1.2.1. Lenguajes compilados	8
1.2.2. Lenguajes interpretados.....	8
1.3. Lenguajes estructurados, orientados a objetos y declarativos	8
1.3.1. Lenguajes estructurados	8
1.3.2. Lenguajes orientados a objetos (OOP)	9
1.3.3. Lenguajes declarativos	9
1.4. Compiladores, intérpretes, transpilers y cross-compilers	9
2. REPRESENTACIÓN DE TIPOS DE DATOS	11
2.1. Tipos de datos básicos y estructuras	11
2.1.1. Tipos de datos básicos (primitivos).....	11
2.1.2. Estructuras de datos.....	11
2.2. Sistema binario	13
2.2.1. Representación de números en binario	13
2.3. Sistema hexadecimal	14
3. OPERADORES	16
3.1. Operadores aritméticos	16
3.2. Operadores relacionales	16
3.3. Operadores lógicos y de asignación	16
3.3.1. Operadores lógicos	16
3.3.2. Operadores de asignación	17

Significado de los iconos que aparecen dentro de los TEMAS:



Examen



Importante



Recordatorio



Atención

4. INSTRUCCIONES CONDICIONALES	18
4.1. Estructuras if, else y switch	18
4.1.1. if y else	18
4.1.2. else if	18
4.1.3. switch	19
4.2. Condicionales anidadas y múltiples	19
4.2.1. Condicionales anidadas	19
4.2.2. Condiciones múltiples (con operadores lógicos)	19
5. BUCLES Y RECURSIVIDAD	20
5.1. Bucles for, while, do...while	20
5.1.1. Bucle for	20
5.1.2. Bucle while	20
5.1.3. Bucle do...while	21
5.2. Llamadas recursivas y condiciones de parada	21
6. PROCEDIMIENTOS, FUNCIONES Y PARÁMETROS.....	23
6.1. Declaración y uso de funciones	23
6.2. Paso de parámetros: por valor y por referencia	24
6.3. Variables locales y globales	24
6.3.2. Variables globales.....	25
6.4. Sobrecarga de funciones (function overloading)	25
7. VECTORES Y REGISTROS	26
7.1. ARRAYS UNIDIMENSIONALES Y MULTIDIMENSIONALES	26
7.1.2. Arrays multidimensionales	27
7.2. Registros o estructuras de datos personalizados	27
7.2.1. Estructuras de datos contiguas	28
7.2.2. Estructuras de datos no contiguas	29
7.2.3. TAD (Tipo Abstracto de Dato)	30
7.2.4. Métodos de acceso en estructuras de datos	31
7.2.5. Algoritmos de ordenación clásicos	32
7.3. Árboles binarios	33

7.3.1. Grado de un árbol	33
7.3.2. Tipos de recorrido en profundidad	33
8. ESTRUCTURA DE UN PROGRAMA	35
8.1. Partes principales de la estructura de un programa.....	35
8.2. Algoritmos.....	36
8.2.1. Características de un algoritmo.....	36
8.2.2. Representación de un algoritmo	37
8.2.3. Tipos de algoritmos según su propósito.....	37

Significado de los iconos que aparecen dentro de los TEMAS:



Examen



Importante



Recordatorio



Atención

1. LENGUAJES DE PROGRAMACIÓN

Los lenguajes de programación son herramientas que permiten expresar de forma estructurada y precisa instrucciones que un ordenador puede entender y ejecutar.

1.1. TIPOS DE LENGUAJES

Los lenguajes de programación se pueden clasificar en función de su nivel de abstracción respecto al hardware en: **lenguajes de bajo nivel y lenguajes de alto nivel.**

1.1.1. Lenguajes de bajo nivel

Son aquellos más cercanos al lenguaje máquina (códigos binarios que entiende directamente el procesador).

- Permiten **un control total del hardware** y son extremadamente **eficientes en ejecución**.
- Son **difíciles de escribir, leer y mantener**, ya que requieren conocer detalles técnicos del funcionamiento del sistema.
- Se utilizan principalmente para:
 - » Desarrollo de controladores de dispositivos.
 - » Programación de sistemas operativos.
- **Ejemplo:** Lenguaje ensamblador (Assembly), que traduce directamente instrucciones a operaciones del procesador.

1.1.2. Lenguajes de alto nivel

Su sintaxis y estructura están diseñadas para ser más cercanas al lenguaje humano, lo que facilita su comprensión y su aprendizaje.

- Son **independientes del hardware**, lo que significa que el mismo programa puede ejecutarse en distintos sistemas con poca o ninguna modificación.
- Permiten **mayor productividad** al desarrollador, ya que abstraen detalles del funcionamiento interno del hardware.
- **Ejemplos:** Python, Java, C++, JavaScript, C#.

1.2. LENGUAJES COMPILADOS E INTERPRETADOS

Otra clasificación importante de los lenguajes de programación se basa en **cómo se ejecutan los programas** escritos en ellos.

1.2.1. Lenguajes compilados

El código fuente (lo que escribe el programador) se traduce completamente a código máquina mediante un compilador.

- El resultado es un **archivo ejecutable** que puede ejecutarse de manera independiente, sin necesidad del código fuente o del compilador.
- **Ventajas:**
 - » **Mayor rendimiento:** los programas se ejecutan más rápido.
 - » **Mayor optimización:** se aprovechan mejor los recursos del sistema.
- **Desventajas:**
 - » **Menor flexibilidad:** para hacer cambios es necesario recompilar.
- **Ejemplos de lenguajes compilados:** C, C++, Go, Rust.

1.2.2. Lenguajes interpretados

El código fuente se traduce y ejecuta línea por línea mediante un intérprete en tiempo real.

- No se genera un archivo ejecutable intermedio.
- **Ventajas:**
 - » **Mayor flexibilidad** para pruebas rápidas y desarrollo ágil.
 - » **Facilidad para detectar errores durante la ejecución.**
- **Desventajas:**
 - » **Menor rendimiento** comparado con lenguajes compilados, porque cada instrucción se traduce sobre la marcha.
- **Ejemplos de lenguajes interpretados:** Python, JavaScript, PHP, Ruby.

1.3. LENGUAJES ESTRUCTURADOS, ORIENTADOS A OBJETOS Y DECLARATIVOS

Los lenguajes de programación pueden clasificarse según su **paradigma**, es decir, según la forma en que el programador estructura el código y resuelve los problemas. A continuación, se describen los tres principales paradigmas: **estructurado**, **orientado a objetos** y **declarativo**.

1.3.1. Lenguajes estructurados

Los lenguajes estructurados se basan en una programación lógica, clara y secuencial. En lugar de usar instrucciones de salto desordenadas como GoTo, estos lenguajes proponen un control del flujo de ejecución mediante estructuras bien definidas. **Este paradigma es ideal para programas que siguen una lógica paso a paso.**

- Se basan en el uso de **estructuras de control** claramente definidas:
 - » Condicionales: if, else, switch
 - » Bucles: for, while, do...while
- El flujo del programa sigue una **secuencia lógica, ordenada y predecible**.
- Fomentan una programación **modular y legible**.

- Muy utilizados en educación y programación de sistemas.
- **Ejemplos:** C, Pascal, Ada.

1.3.2. Lenguajes orientados a objetos (OOP)

Los lenguajes orientados a objetos permiten organizar el software en torno a entidades llamadas **objetos**, que combinan tanto datos como funciones. Este paradigma busca reflejar el mundo real mediante clases y relaciones entre ellas, facilitando la reutilización del código y su mantenimiento.

- Basado en la creación de **objetos** (instancias de clases) que agrupan atributos y métodos.
- Principios clave:
 - » **Encapsulamiento:** oculta la implementación interna.
 - » **Herencia:** permite extender funcionalidades.
 - » **Polimorfismo:** adapta métodos según el contexto.
- **Ejemplos:** Java, C++, Python, C#.

1.3.3. Lenguajes declarativos

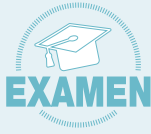
En los lenguajes declarativos, el programador se enfoca en **qué se desea conseguir, en lugar de detallar cómo se debe hacer**. Este paradigma delega en el lenguaje la responsabilidad de determinar la mejor forma de alcanzar el resultado.

- Se describe el resultado esperado, no los pasos.
- Tipos principales:
 - » **Funcionales** (Haskell): usan funciones sin efectos secundarios.
 - » **Lógicos** (Prolog): basados en reglas y hechos.
 - » **Consultas** (SQL): extraen o manipulan información.
- **Ejemplos:** SQL, Prolog, Haskell.

1.4. COMPILADORES, INTÉRPRETES, TRANSPILERS Y CROSS-COMPILERS

En el desarrollo de software, los lenguajes de programación necesitan traducirse a código ejecutable por la máquina. Para ello existen distintos tipos de herramientas:

- **Intérprete:** Ejecuta el código línea a línea directamente, sin generar un archivo ejecutable.
 - Útil para pruebas rápidas y desarrollo interactivo.
 - Ejemplo: Python, JavaScript.
- **Compilador:** Traduce el código fuente a código máquina o binario.
 - Produce un archivo ejecutable independiente.



- **Transpilador (source-to-source compiler):** Traduce código fuente de un lenguaje a otro de nivel similar.
 - No es un híbrido entre compilador e intérprete.
 - Ejemplo: TypeScript → JavaScript, Babel (JavaScript moderno → antiguo).
- **Compilador cruzado (cross compiler):** Genera código ejecutable para una plataforma distinta a aquella en la que se ejecuta el compilador.
 - Muy usado en sistemas embebidos y desarrollo multiplataforma.



- **Bytecode:** Código intermedio que no es código máquina, pero tampoco es código fuente.
 - Se genera, por ejemplo, al compilar un programa Java (.class), y es interpretado por la máquina virtual (JVM).
 - Permite **portabilidad** entre sistemas.
- **Lenguajes de programación por generación**
 - Lenguajes de tercera generación (3GL): C, C++, Java, Python...
 - Lenguajes de segunda generación (2GL): ensamblador.
 - Lenguajes de primera generación (1GL): código máquina binario.

2. REPRESENTACIÓN DE TIPOS DE DATOS

Los datos en programación deben representarse de forma que puedan ser **procesados por los ordenadores**, los cuales operan a nivel binario. Por ello, es importante conocer tanto los **tipos de datos básicos y estructuras** que se usan en los lenguajes de programación, como los **sistemas de numeración** que permiten representar esa información internamente.

2.1. TIPOS DE DATOS BÁSICOS Y ESTRUCTURAS

En los lenguajes de programación, los **tipos de datos** son fundamentales porque permiten representar, almacenar y manipular distintos tipos de información. Se clasifican en dos grandes grupos: **tipos de datos básicos (o primitivos)** y **estructuras de datos compuestas o complejas**.

2.1.1. Tipos de datos básicos (primitivos)

Son los tipos de datos más simples y elementales. Suelen estar integrados en todos los lenguajes y representan unidades mínimas de información.

- **Enteros (int)**: Representan números enteros, es decir, **sin decimales**.
 - » Tamaño y rango dependen del lenguaje y la arquitectura (por ejemplo, int, short, long, long long).
- **Reales o flotantes (float, double)**: Representan números **con parte decimal**.
 - » **float**: menor precisión y menor uso de memoria.
 - » **double**: mayor precisión (usado en cálculos científicos).
- **Carácter (char)**: Representa **un único carácter**: letra, número o símbolo.
 - » Ejemplos: 'A', '7', '#'
 - » Se almacenan internamente como códigos numéricos (por ejemplo, ASCII o Unicode).
 - » En muchos lenguajes, un char ocupa 1 byte (8 bits).
- **Booleano (bool)**: Representa una **condición lógica**: solo puede ser true (verdadero) o false (falso).
 - » Muy usado en estructuras de control (if, while).
 - » Suelen ocupar 1 bit o un byte, dependiendo del lenguaje.

2.1.2. Estructuras de datos

Las **estructuras de datos** permiten organizar y almacenar conjuntos de información de forma eficiente, para facilitar su acceso, manipulación y procesamiento. A partir de los tipos de datos básicos, se construyen estructuras más complejas que permiten representar datos más elaborados.

- **Cadenas de texto (string):** Una cadena es una secuencia ordenada de caracteres (letras, dígitos, símbolos...).
 - Internamente, cada carácter se representa mediante un código numérico (ASCII, Unicode).
 - Pueden tener operaciones como: concatenación, búsqueda, extracción de subcadenas, comparación, etc.
 - **Ejemplo:** "Hola mundo", "1234", "admin@correo.com"
- **Arrays o vectores: Un array (o vector) es una colección ordenada de elementos del mismo tipo.**
 - Cada elemento se accede mediante un **índice** (posición numérica).
 - El número de elementos puede ser **fijo** (array estáticos) o **variable** (dinámicos, dependiendo del lenguaje).
 - **Ejemplo en C:** `int edades[3] = {18, 25, 30};`
- **Registros o estructuras (struct):** Agrupan **varios campos de distinto tipo** bajo un mismo nombre.
 - Representan objetos reales con múltiples propiedades.
 - Cada campo (o atributo) tiene un nombre propio y puede ser de tipo distinto.
 - **Ejemplo en C:**

```
struct Alumno {  
    char nombre[20];  
    int edad;  
    float nota;  
};
```

- **Estructuras dinámicas avanzadas:** Estas estructuras permiten una gestión más flexible de los datos, especialmente cuando la cantidad o el orden cambia con frecuencia.
 - **Listas enlazadas:** cada elemento apunta al siguiente (y a veces al anterior). Permiten inserciones y eliminaciones dinámicas.
 - **Pilas (stacks):** estructura LIFO (Last In, First Out). Solo se accede al último elemento insertado.
 - **Colas (queues):** estructura FIFO (First In, First Out). El primero en entrar es el primero en salir.
 - **Árboles:** estructuras jerárquicas con nodos (como un organigrama). Muy usados en bases de datos, compiladores y sistemas de archivos.
 - **Grafos:** conjunto de nodos conectados entre sí. Se utilizan en redes, mapas y algoritmos de búsqueda.
- **Un diccionario** (también conocido como mapa o tabla hash) es un tipo abstracto de datos que almacena pares clave-valor. A diferencia de estructuras como listas o pilas, no garantiza un orden específico en la disposición de sus elementos. Esto significa que los elementos pueden almacenarse y recuperarse sin seguir un orden determinado.

- **Mapa:** Aunque similar al diccionario, en algunos lenguajes de programación, como Java, existen implementaciones de mapas que mantienen un orden específico, como TreeMap, que ordena las claves de forma natural.

2.2. SISTEMA BINARIO

El sistema binario es fundamental puesto que constituye la **base del funcionamiento de los ordenadores y dispositivos digitales**. Toda la información que se procesa, almacena o transmite se codifica utilizando únicamente dos símbolos: **0 y 1**, llamados **bits**.

2.2.1. Representación de números en binario

- Los números decimales (base 10) se convierten a binario (base 2) expresando el número como una suma de potencias de 2.
- En binario, cada posición de un número representa una potencia de 2 (2^0 , 2^1 , 2^2 , 2^3 , etc.), comenzando desde la derecha.
- **Representación:** cada número en binario es una secuencia de bits.
- Las operaciones lógicas y aritméticas a bajo nivel (AND, OR, NOT, suma binaria) se realizan en binario.

Para la conversión de binario a decimal, y viceversa, se propone el uso de la siguiente tabla. En el ejemplo que se detalla a continuación se expone el funcionamiento de la misma.

9	8	7	6	5	4	3	2	1	0
2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1

DECIMAL - BINARIO:

Vamos poniendo 1 o 0 debajo de cada cifra hasta obtener la suma en decimal que estamos buscando.

BINARIO - DECIMAL:

Vamos poniendo 1 o 0 debajo de cada cifra y después sumamos las cifras decimales.

- Por ejemplo:

» Decimal 5 → Binario **101**

2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1
0	0	0	0	0	0	0	1	0	1

» Binario **1010 → 8 + 2 = 10**

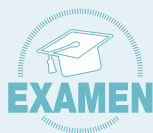
2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1
0	0	0	0	0	0	1	0	1	0



- **Bit:** unidad mínima de información, puede ser 0 o 1.
- **Byte:** conjunto de 8 bits. Unidad básica de almacenamiento de datos. Un byte puede representar 256 valores diferentes (de 0 a 255).
- **Nibbles:** conjunto de 4 bits. Se utiliza mucho en representaciones hexadecimales.
- El **agrupamiento de 8 bits forma un byte**, unidad básica de almacenamiento.

2.3. SISTEMA HEXADECIMAL

El **sistema hexadecimal** es un sistema de numeración en base 16, muy utilizado en informática por su correspondencia directa con grupos de 4 bits (un nibble).



- Utiliza los dígitos:
0–9 para los valores decimales 0 a 9
A–F para los valores decimales 10 a 15

El sistema hexadecimal se emplea habitualmente para representar:

- **Direcciones de memoria.**
- **Colores en HTML/CSS.** Ej.: #FF0000 (rojo).
- **Códigos de caracteres (ASCII, Unicode).**

Para realizar el cálculo HEXADECIMAL:

1. OPCIÓN 1: Para convertir un número **decimal a hexadecimal**, se divide el número decimal sucesivamente por 16 y se toman los residuos como dígitos hexadecimales. El proceso se repite hasta que el cociente es 0.
2. OPCIÓN 2: Convierte el número decimal en binario, divídelo en grupos de 4, busca la equivalencia en la tabla de equivalencia siguiente.

175 decimal = 10101111 b) => 1010 y 1111 = AF

"0000"	0	0
"0001"	1	1
"0010"	2	2
"0011"	3	3
"0100"	4	4
"0101"	5	5
"0110"	6	6
"0111"	7	7
"1000"	8	8
"1001"	9	9
"1010"	A	10
"1011"	B	11
"1100"	C	12
"1101"	D	13
"1110"	E	14
"1111"	F	15

3. HEXADECIMAL-DECIMAL: Convierte cada cifra en hexadecimal en binario, siguiente la tabla. Después realiza la conversión binario-decimal habitual

- **Ejemplos:**
 - » Decimal 10 → Hexadecimal **A**
 - » Decimal 255 → Hexadecimal **FF**
 - » Binario 11111111 → Hexadecimal **FF**

3. OPERADORES

Los **operadores** son símbolos que permiten realizar operaciones entre variables o valores. En programación, se dividen según el tipo de operación que realizan: aritmética, comparación (relacional), lógica o asignación.

3.1. OPERADORES ARITMÉTICOS

Se utilizan para realizar **operaciones matemáticas** básicas entre valores numéricos.

Operador	Significado	Ejemplo	Resultado
+	Suma	5 + 2	7
-	Resta	9 - 4	5
*	Multiplicación	3 * 4	12
/	División	10 / 2	5.0
%	Módulo (resto)	10 % 3	1
**	Potencia (en algunos lenguajes, como Python)	2 ** 3	8
++	Incrementa el valor en 1	1++	2
--	Decremento el valor en 1	2--	1

3.2. OPERADORES RELACIONALES

También llamados operadores de comparación, se utilizan para **comparar dos valores**. El resultado siempre es un valor booleano: true o false.

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	true
!=	Distinto de	4 != 3	true
>	Mayor que	8 > 6	true
<	Menor que	3 < 2	false
>=	Mayor o igual que	5 >= 5	true
<=	Menor o igual que	7 <= 10	true

3.3. OPERADORES LÓGICOS Y DE ASIGNACIÓN

3.3.1. Operadores lógicos

Permiten realizar **operaciones booleanas**. Se usan comúnmente en condiciones (if, while...).

Operador	Significado
&&/and	Y lógico
/or	O lógico
!/not	Negación

Nota: La sintaxis puede variar según el lenguaje (por ejemplo, en Python se usan and, or, not).

3.3.2. Operadores de asignación

Sirven para **asignar valores a variables**, o modificar su valor existente de forma abreviada.

Operador	Significado	Ejemplo	Equivale a
=	Asignación simple	x = 5	—
+=	Suma y asigna	x += 2	x = x + 2
-=	Resta y asigna	x -= 1	x = x - 1
*=	Multiplica y asigna	x *= 3	x = x * 3
/=	Divide y asigna	x /= 2	x = x / 2
%=	Módulo y asigna	x %= 4	x = x % 4
**=	Potencia y asigna (Python)	x **= 2	x = x ** 2
//=	División entera (Python)	x //= 3	x = x // 3
&=	AND bit a bit y asigna	x &= 1	x = x & 1
^=	XOR bit a bit y asigna	x ^= 3	x = x ^ 3
<<=	Desplaza a la izquierda	x <<= 1	x = x << 1
>>=	Desplaza a la derecha	x >>= 1	x = x >> 1

4. INSTRUCCIONES CONDICIONALES

Las instrucciones condicionales permiten que un programa tome decisiones durante su ejecución. Evaluando una condición lógica (**una expresión que da como resultado true o false**), se determina qué bloque de código debe ejecutarse.

4.1. ESTRUCTURAS IF, ELSE Y SWITCH

4.1.1. if y else

Es la forma más común de tomar decisiones en un programa. **Se ejecuta un bloque de código si se cumple una condición, y otro bloque en caso contrario.**

Sintaxis básica:

```
if (condición) {  
    // código si la condición es verdadera  
} else {  
    // código si la condición es falsa  
}
```

Ejemplo:

```
if (edad >= 18) {  
    printf("Eres mayor de edad.\n");  
} else {  
    printf("Eres menor de edad.\n");  
}
```

4.1.2. else if

Permite evaluar varias condiciones encadenadas.

Ejemplo:

```
if (nota >= 9) {  
    printf("Sobresaliente");  
} else if (nota >= 7) {  
    printf("Notable");  
} else if (nota >= 5) {  
    printf("Aprobado");  
} else {  
    printf("Suspenso");  
}
```

4.1.3. switch

Es una alternativa al uso de múltiples if-else cuando se evalúa una misma variable frente a diferentes valores.

Ejemplo:

```
switch (opcion) {
    case 1:
        printf("Has elegido café.\n");
        break;
    case 2:
        printf("Has elegido té.\n");
        break;
    default:
        printf("Opción no válida.\n");
}
```

4.2. CONDICIONALES ANIDADAS Y MÚLTIPLES

4.2.1. Condicionales anidadas

Una estructura condicional puede contener otra dentro de su bloque. Se utilizan cuando es necesario evaluar subdecisiones.

Ejemplo:

```
if (usuario == "admin") {
    if (clave == "1234") {
        printf("Acceso permitido");
    } else {
        printf("Contraseña incorrecta");
    }
}
```

4.2.2. Condiciones múltiples (con operadores lógicos)

Se pueden combinar varias condiciones utilizando operadores lógicos como && (AND), || (OR), o ! (NOT).

Ejemplo:

```
if (edad > 18 && nacionalidad == "española") {
    printf("Puedes votar en España");
}
```

5. BUCLES Y RECURSIVIDAD

Los bucles y la recursividad permiten ejecutar una misma porción de código varias veces. Se utilizan para automatizar tareas repetitivas o recorrer estructuras de datos.

5.1. BUCLES FOR, WHILE, DO...WHILE

Los bucles permiten repetir un conjunto de instrucciones mientras se cumpla una condición.

5.1.1. Bucle for

Se utiliza cuando se conoce el número exacto de repeticiones. Tiene tres partes: inicialización, condición y actualización.

Sintaxis básica:

```
for (inicialización; condición; incremento) {  
    // Bloque de instrucciones  
}
```

Ejemplo: Este bucle imprimirá los números del 0 al 4.

```
for (int i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

5.1.2. Bucle while

Repite las instrucciones **mientras** se cumpla una condición. La condición se evalúa antes de ejecutar el cuerpo.

Sintaxis básica:

```
while (condición) {  
    // Bloque de instrucciones  
}
```

Ejemplo: Este bucle imprimirá los números del 0 al 4

```
int i = 0;  
while (i < 5) {  
    printf("%d\n", i);  
    i++;  
}
```

5.1.3. Bucle do...while

Es similar a while, pero la **condición se evalúa al final**, por lo que **el bloque se ejecuta al menos una vez**.

Sintaxis básica:



```
do {  
  
    // Bloque de instrucciones  
  
} while (condición);
```

Ejemplo: Este bucle imprimirá los números del 0 al 4

```
int i = 0;  
do {  
    printf("%d\n", i);  
    i++;  
} while (i < 5);
```

5.2. LLAMADAS RECURSIVAS Y CONDICIONES DE PARADA

La **recursividad** es una técnica de programación en la que una función se llama a sí misma para resolver un problema. Es especialmente útil cuando un problema puede dividirse en subproblemas del mismo tipo, permitiendo una solución elegante y muchas veces más intuitiva que utilizando bucles iterativos.

Conceptos clave

Una función recursiva debe tener **dos componentes fundamentales** para que funcione correctamente:

1. Condición de parada (caso base):

Es la condición que indica cuándo dejar de llamar a la función a sí misma. Sin esta condición, la recursión continuaría indefinidamente, provocando un desbordamiento de pila (*stack overflow*).

2. Llamada recursiva:

Es la instrucción dentro de la función que hace que ésta se vuelva a invocar con un valor diferente (normalmente más cercano al caso base). El objetivo es reducir el problema progresivamente hasta llegar al caso base.

Estructura básica de una función recursiva:

Una función recursiva **se llama a sí misma** para resolver un problema. Pero para que esto no cause una ejecución infinita, **debe haber una condición que indique cuándo detenerse**. Esa estructura general se puede dividir en dos partes fundamentales:

1. Caso base o condición de parada → evita llamadas infinitas.
2. Llamada recursiva → se llama a sí misma con datos reducidos.

```
tipo nombre_funcion(tipo parametro) {  
    if (condicion_de_parada)  
        return valor_base;  
    else  
        return llamada_recursiva_con_valor_mas_simple;  
}
```

- **parametro** → es el valor que recibe la función y que se irá modificando en cada llamada.
- **condicion_de_parada** → una condición que indica cuándo hemos llegado al caso más simple del problema.
- **valor_base** → el resultado directo cuando se cumple la condición de parada.
- **llamada_recursiva_con_valor_mas_simple** → es la llamada a la misma función, pero con un parámetro modificado (por ejemplo, n-1).

Ejemplo (factorial):

```
int factorial(int n) {  
    if (n == 0) return 1;           // Caso base  
    else return n * factorial(n - 1); // Llamada recursiva  
}
```

6. PROCEDIMIENTOS, FUNCIONES Y PARÁMETROS

Una **función** o **procedimiento** es un bloque de código reutilizable que realiza una tarea específica. Permiten **modularizar** el programa y mejorar la legibilidad, el mantenimiento y la reutilización.

Elementos de una función:

Una función consta de:

1. **Nombre** → Identificador que se usa para llamarla.
2. **Parámetros** → Datos de entrada que la función necesita para realizar su tarea (pueden ser cero o varios).
3. **Tipo de retorno** → Indica el tipo de dato que la función devuelve (por ejemplo, int, float, char, etc.). Si no devuelve nada, se usa void.
4. **Cuerpo o bloque de instrucciones** → Contiene el código que se ejecuta cuando se llama a la función.
5. **Sentencia return** → Opcional. Devuelve un resultado al punto donde fue llamada (solo si el tipo de retorno no es void).

6.1. DECLARACIÓN Y USO DE FUNCIONES

Una **función** tiene un nombre, puede recibir datos de entrada (parámetros) y, opcionalmente, devuelve un valor.

Estructura general de una función:

```
tipo_retorno nombre_funcion(tipo parametro1, tipo parametro2,
...) {
    // instrucciones
    return valor; // si devuelve algo
}
```

Procedimiento: Función sin retorno

Un **procedimiento** es una función que **no devuelve ningún valor**. En C, se define con el tipo void.

```
void mostrarSaludo() {
    printf("¡Hola, bienvenido!\n");
}
```

Y se llama así:

```
int main() {
    mostrarSaludo(); // Llamada
    return 0;
}
```

6.2. PASO DE PARÁMETROS: POR VALOR Y POR REFERENCIA

Cuando se llama a una función, se le pueden pasar datos o argumentos. La **forma en que se pasan esos datos** puede influir en el comportamiento de la función y en si los valores originales pueden o no modificarse. Existen dos formas principales de pasar parámetros:

6.2.1. Paso por valor

- Se pasa **una copia del valor** del argumento original.
- La función **trabaja con una copia**, no con la variable original.
- **Cualquier cambio** que se realice dentro de la función **no afecta** al valor original fuera de ella.

Ejemplo:

```
función duplicar(x)
    x = x * 2
fin función
variable a = 10
llamar a duplicar(a)
// a sigue valiendo 10 después de la llamada
```

6.2.2. Paso por referencia

- Se pasa **una referencia** o **acceso directo** al dato original (por ejemplo, su dirección de memoria o su identificador en el contexto).
- Los **parámetros por referencia** se pueden emplear como parámetros de **entrada/salida**.
- **Los cambios realizados dentro de la función afectan directamente al valor original**.

Ejemplo:

```
función duplicar_referencia(referencia x)
    x = x * 2
fin función
variable a = 10
llamar a duplicar_referencia(a)
// a ahora vale 20 después de la llamada
```

6.3. VARIABLES LOCALES Y GLOBALES



Las **variables** son espacios en memoria que almacenan datos y tienen un nombre asociado. En un programa, su **ámbito o visibilidad** depende de **dónde se declaran**, lo cual influye directamente en **dónde y cuándo se pueden usar**. Existen dos tipos principales de variables según su ámbito: LOCALES y GLOBALES.

6.3.1. Variables locales

- Se **declaran dentro de una función, bloque o estructura de control** (como un bucle o una condición).
- Solo **existen y son accesibles** durante la ejecución de esa función o bloque.
- Se **crean al entrar** en la función y se **eliminan al salir**.
- **No interfieren ni pueden ser usadas** desde otras partes del programa.

Ejemplo:

```
función ejemplo()
    definir variable local = 10
    // solo se puede usar dentro de esta función
fin función
```

6.3.2. Variables globales

- Se **declaran fuera de cualquier función o bloque**, normalmente al principio del programa.
- Están **disponibles para todo el programa**, es decir, **pueden ser leídas y modificadas desde cualquier función**.
- Su valor **se conserva** durante toda la ejecución del programa.

Ejemplo:

```
definir variable global = 5
función mostrar()
    imprimir global // accesible desde cualquier parte
fin función
```

Comparativa locales-globales

Aspecto	Variables locales	Variables globales
Ámbito	Dentro de su función o bloque	En todo el programa
Tiempo de vida	Mientras se ejecuta la función	Durante toda la ejecución del programa
Acceso desde otras funciones	No	Sí
Riesgo de interferencias	Bajo	Alto, si se modifican en muchas partes
Uso recomendado	Datos temporales o intermedios	Datos compartidos entre funciones

6.4. SOBRECARGA DE FUNCIONES (FUNCTION OVERLOADING)

En algunos lenguajes como C++, es posible definir **varias funciones con el mismo nombre** si tienen diferente **número o tipo de parámetros**. Esto se llama **sobrecarga de funciones**.

Ejemplo en C++: Ambas funciones se llaman SUMar, pero C++ selecciona la correcta según los argumentos al momento de la llamada

```
int sumar(int a, int b);
float sumar(float x, float y);
```

No se debe confundirse con **sobrescritura** (override), que se refiere a redefinir funciones heredadas en programación orientada a objetos.

7. VECTORES Y REGISTROS

Los vectores o arrays y los registros (también conocidos como estructuras) son **estructuras de datos** fundamentales en programación. Permiten almacenar y organizar grandes cantidades de información de forma eficiente.

7.1. ARRAYS UNIDIMENSIONALES Y MULTIDIMENSIONALES

Un **array** es una estructura de datos que permite almacenar **múltiples valores del mismo tipo** en posiciones consecutivas de memoria. Cada valor se llama **elemento** y se accede a él mediante un **índice**.

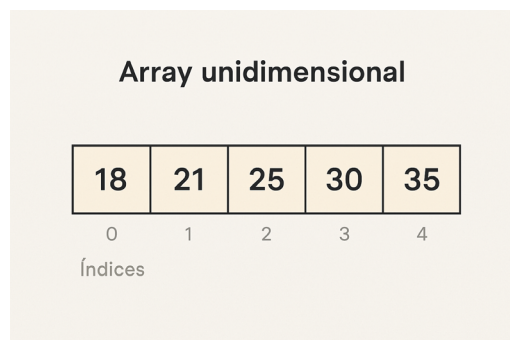
7.1.1. Arrays unidimensionales (vectores)

Un array unidimensional es una **secuencia lineal** de elementos del mismo tipo. Cada elemento se identifica por su **posición (índice)**, que comienza normalmente en **0**.

Características:

- Todos los elementos son del mismo tipo (por ejemplo, enteros, reales, caracteres, etc.).
- El tamaño del array debe definirse (según el lenguaje, puede ser fijo o dinámico).
- El acceso a los elementos es directo mediante el índice: `array[i]`.

Ejemplo conceptual (vector de enteros con 5 elementos): Si se accede a **`array[2]`**, obtenemos el valor **25**.



7.1.2. Arrays multidimensionales

Un array multidimensional es un **array que contiene otros arrays**. El más común es el **array bidimensional**, que se utiliza para representar **tablas o matrices** (filas y columnas).

- **Array bidimensional:** Se organiza como una tabla de filas × columnas, y se accede mediante dos índices: uno para la fila y otro para la columna.

Ejemplo:

- `matriz[0][1]` → accede al valor en la primera fila, segunda columna → valor: 2
- `matriz[1][2]` → accede a la segunda fila, tercera columna → valor: 6

		Columnas		
Filas	Posiciones	[0]	[1]	[2]
	[0]	1	2	3
	[1]	1	5	6
	[2]	4	5	6

- **Arrays de más dimensiones:** Pueden existir arrays de 3 o más dimensiones (por ejemplo, para representar cubos de datos, como en aplicaciones científicas o gráficas). Cada dimensión añade un nivel de indexación.

array3D[x][y][z]

Importante:

- Los índices deben estar **dentro del rango** del array. Acceder a una posición fuera de ese rango puede provocar errores o comportamientos indeseados.
- El tamaño de los arrays **estáticos** debe conocerse en tiempo de compilación (según el lenguaje).
- En muchos lenguajes modernos, también existen **arrays dinámicos**, listas o colecciones que permiten cambiar de tamaño en tiempo de ejecución.

7.2. REGISTROS O ESTRUCTURAS DE DATOS PERSONALIZADOS

En programación, un **registro** (también llamado **estructura**, *record*, o *struct*) es una **colección de datos agrupados** bajo un mismo nombre, donde **cada elemento (campo)** puede ser de **tipo distinto**. Esto permite modelar objetos del mundo real de forma más natural y estructurada, combinando datos heterogéneos en una única unidad lógica.

Las estructuras permiten:

- Organizar datos relacionados entre sí (como la información de una persona, producto o vehículo).
- Reunir diferentes tipos de datos en una sola entidad (por ejemplo: cadenas, números enteros y reales).

Estructura de una estructura: Cada campo (o atributo) representa una característica del objeto que se está modelando.

```
estructura NombreEstructura
    tipo1 campo1
    tipo2 campo2
    ...
fin estructura
```

Ejemplo: estructura para representar una persona

```
estructura Persona
    cadena nombre
    entero edad
    real altura
fin estructura
variable alumno de tipo Persona
alumno.nombre = "Lucía"
alumno.edad = 20
alumno.altura = 1.65
```

Acceso a los campos: Para acceder a los valores de una estructura, se utiliza el operador punto (.)

```
estructura.campo
```

En el ejemplo anterior:

- `alumno.nombre` → devuelve "Lucía"
- `alumno.edad` → devuelve 20

7.2.1. Estructuras de datos contiguas

Una estructura de datos se considera **contigua** cuando **todos sus elementos se almacenan en posiciones consecutivas de memoria**. Esto permite un acceso rápido y directo a cualquier elemento mediante su índice, ya que la posición de cada uno puede calcularse fácilmente a partir de la posición inicial.

- **Arrays (vectores)**
 - Conjunto de elementos **homogéneos** (del mismo tipo).
 - Se accede por índice: `array[i]`.
 - El acceso a cualquier posición es en **tiempo constante (O(1))**.
 - Tamaño generalmente fijo (salvo en arrays dinámicos de lenguajes modernos).
- **Cadenas de caracteres (strings)**
 - En muchos lenguajes, las cadenas son implementadas como arrays de caracteres.
 - Cada carácter ocupa una posición contigua en memoria.

- **Arrays multidimensionales**

- Representan matrices (bidimensionales o más).
- Aunque se acceden con múltiples índices, se almacenan como una **secuencia lineal** en memoria.
- Organización **fila a fila (row-major)** o **columna a columna (column-major)** dependiendo del lenguaje.

7.2.2. Estructuras de datos no contiguas

Se dice que una estructura de datos es **no contigua** cuando **sus elementos no están almacenados en posiciones consecutivas de memoria**. En lugar de depender de la cercanía física en la memoria, los elementos están **vinculados entre sí mediante referencias o punteros**.

Estas estructuras permiten mayor flexibilidad en operaciones como inserción o eliminación de elementos, pero pueden requerir más tiempo de acceso y gestión de memoria más compleja.

- **Listas enlazadas (listas simples, dobles, circulares)**

- Cada elemento (nodo) contiene:
 - » Un **dato**.
 - » Un **puntero** o referencia al siguiente (y/o anterior) nodo.
- Los nodos pueden estar ubicados en cualquier parte de la memoria.
- Muy eficientes para inserciones y eliminaciones dinámicas.

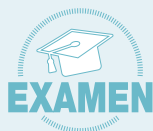
- **Árboles (binarios, AVL, B-trees...)**

- Estructura jerárquica.
- Cada nodo contiene:
 - » Un valor.
 - » Referencias a sus nodos hijos (izquierdo, derecho).
- Muy utilizados en búsquedas, jerarquías y organización eficiente de datos.

- **Grafos**

- Conjunto de nodos (vértices) conectados por aristas.
- Las conexiones entre elementos no siguen una estructura secuencial ni lineal.
- Representados mediante listas de adyacencia o matrices dispersas.

7.2.3. TAD (Tipo Abstracto de Dato)



Un **TAD (Tipo Abstracto de Dato)** es una **descripción lógica de una estructura de datos** que **define su comportamiento desde el punto de vista del usuario**, sin especificar cómo se implementa internamente.

TAD	Descripción	Ejemplo de uso
TAD Lista	Colección ordenada de elementos, con acceso secuencial o por índice.	Listas de nombres, tareas, etc.
TAD Pila	Estructura LIFO (último en entrar, primero en salir).	Control de llamadas, deshacer acciones.
TAD Cola	Estructura FIFO (primero en entrar, primero en salir).	Impresoras, procesos en espera.
TAD Bicola (Deque)	Inserción y eliminación por ambos extremos.	Navegación adelante/atrás.
TAD Árbol	Estructura jerárquica con nodos enlazados.	Representación de jerarquías.
TAD Grafo	Conjunto de nodos conectados por aristas (dirigidas o no).	Redes, mapas, relaciones.
TAD Diccionario (Mapa)	Asocia claves con valores.	Tablas de símbolos, almacenamiento clave-valor.

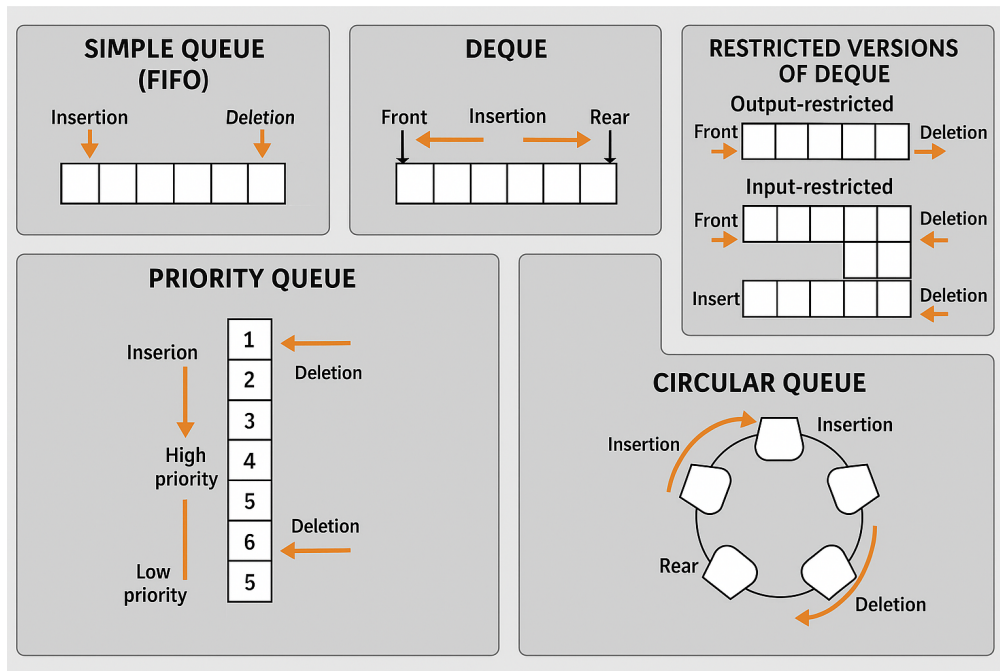
Una **cola** (*queue*) es una estructura de datos lineal que sigue el principio **FIFO (First In, First Out)**: el **primer elemento en entrar es el primero en salir**. Es útil cuando se necesita un procesamiento en orden, como en tareas en cola, impresión o buffers.

- **Cola simple (FIFO).**
 - Inserción: al final de la cola.
 - Eliminación: al inicio de la cola.
 - No permite insertar ni eliminar en posiciones intermedias.



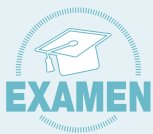
- **Bicola (double-ended queue o deque):** Una bicola es una variante de la cola que permite inserciones y eliminaciones por ambos extremos (inicio y final). Es más flexible que una cola simple.
 1. **Cola de salida restringida:** las inserciones se permiten por ambos extremos, y las eliminaciones solo por un extremo
 2. **Cola de entrada restringida:** las inserciones se permiten solo por un extremo y las eliminaciones por ambos extremos

- **Cola de prioridad:** Una cola de prioridad permite que cada elemento tenga un valor de prioridad, y los elementos con mayor prioridad se extraen antes que otros, independientemente del orden en que se insertaron.
- **Cola circular.** Una cola circular es una variante de la cola simple que reutiliza el espacio libre al principio del array una vez que se han eliminado elementos. **Usa dos punteros: inicio y fin, que se mueven circularmente en un array.**



7.2.4. Métodos de acceso en estructuras de datos

Las estructuras de datos no solo se diferencian por su forma de almacenamiento (contiguo o no contiguo), sino también por la **forma en que se accede y gestiona la información**. Esta forma se conoce como **método de acceso**, y determina **el orden en que los elementos se insertan y se extraen**.



- **FIFO (First In, First Out) — Cola (Queue)**
 - El **primer elemento en entrar es el primero en salir**.
 - Se comporta como una **línea de espera**: el primero en llegar es el primero en ser atendido.
 - Estructura típica: **cola** (queue).
 - **Operaciones principales:**
 - enqueue(): insertar al final.
 - dequeue(): eliminar del frente.

- **LIFO (Last In, First Out) — Pila (Stack)**
 - El **último elemento en entrar es el primero en salir**.
 - Se comporta como una **pila de platos**: el último que se apila es el primero en retirarse.
 - Estructura típica: **pila** (stack).
- **Operaciones principales:**
 - `push()`: insertar al tope.
 - `pop()`: eliminar del tope.
- **Acceso directo o aleatorio (Random Access)**
 - Permite acceder a **cualquier elemento directamente**, sin recorrer otros.
 - Requiere estructuras **contiguas**, como los **arrays**.
 - El acceso se realiza mediante **índices**.



- **Acceso secuencial**
 - El acceso se realiza **elemento a elemento**, en orden.
 - Común en **listas enlazadas** y **archivos secuenciales**.
 - Necesario cuando no hay índices o los datos están dispersos.

7.2.5. Algoritmos de ordenación clásicos

Los algoritmos de ordenación permiten reorganizar los elementos de una estructura de datos (como un array o lista) en un orden determinado (generalmente ascendente o descendente). A continuación se describen los métodos más conocidos, ideales para comprender las bases de la ordenación.

1. **Bubble Sort (ordenación por intercambio directo).** Comparar e intercambiar elementos adyacentes si están en orden incorrecto.
2. **Selection Sort (ordenación por selección).** Encontrar el menor elemento del array y colocarlo en la primera posición; luego repetir con el resto.
3. **Insertion Sort (ordenación por inserción).** Tomar un elemento y ubicarlo en su lugar correcto respecto a los anteriores ya ordenados.
4. **Merge Sort (ordenación por mezcla).** Dividir el array en mitades, ordenar cada mitad recursivamente, y luego fusionarlas.
5. **Quick Sort (ordenación rápida).** Elegir un pivote, dividir el array en elementos menores y mayores, y ordenar recursivamente. **Técnica: divide y vencerás.**
6. **Heap Sort (ordenación por montículos).** Construir un montículo (heap) y extraer sucesivamente el elemento máximo.

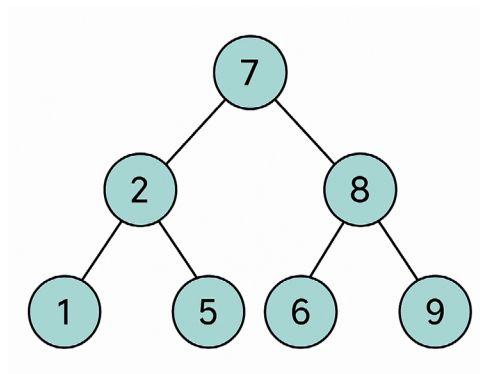
7.3. ÁRBOLES BINARIOS

Un **árbol binario** es una estructura de datos jerárquica en la que cada nodo tiene como máximo **dos hijos**: un **subárbol izquierdo** y un **subárbol derecho**. Los árboles se utilizan para organizar datos de forma eficiente, por ejemplo, en algoritmos de búsqueda, jerarquías y compiladores.

Un **árbol** es una **estructura de datos dinámica no lineal**, compuesta por nodos enlazados. **No es estática porque su tamaño puede crecer o disminuir durante la ejecución del programa.**

Permite dos tipos de recorridos principales:

- **En profundidad** (DFS, *Depth-First Search*): preorden, inorden, postorden.
- **En amplitud** (BFS, *Breadth-First Search*): recorre por niveles usando una cola.



Para procesar o visualizar la información contenida en un árbol binario, existen **varios tipos de recorridos en profundidad** (*depth-first traversal*), que se diferencian en el **orden en que se visitan** los nodos: la raíz, el subárbol izquierdo y el subárbol derecho.

7.3.1. Grado de un árbol

El **grado** de un árbol se refiere al **número máximo de hijos que puede tener un nodo**.

- En un **árbol general**, un nodo puede tener cualquier número de hijos.
- En un **árbol binario**, cada nodo puede tener **como máximo 2 hijos**, por lo que **el grado del árbol binario es 2**, independientemente del número de nodos o su forma.

No debe confundirse con:

- **Altura** del árbol: número de niveles desde la raíz hasta la hoja más profunda.
- **Grado de un nodo**: número de hijos que tiene un nodo específico.

7.3.2. Tipos de recorrido en profundidad

1. Preorden (raíz → izquierdo → derecho)

- Se visita **primero la raíz**, luego el **subárbol izquierdo** y finalmente el **subárbol derecho**.
- Se usa, por ejemplo, para copiar árboles o generar expresiones prefijas.

2. Inorden (izquierdo → raíz → derecho)

- Se visita **primero el subárbol izquierdo**, luego la **raíz** y finalmente el **subárbol derecho**.
- En árboles binarios de búsqueda (ABB), este recorrido devuelve los datos **ordenados ascendentemente**.

3. Postorden (izquierdo → derecho → raíz)

- Se visita **primero el subárbol izquierdo**, luego el **derecho**, y por último la **raíz**.
- Se utiliza, por ejemplo, para eliminar árboles o generar expresiones postfijas (notación polaca inversa).

Ejemplo:



Los recorridos serían:

- **Preorden:** A, B, D, E, C, F
- **Inorden:** D, B, E, A, C, F
- **Postorden:** D, E, B, F, C, A

8. ESTRUCTURA DE UN PROGRAMA

Un programa informático es un conjunto de instrucciones organizadas de forma lógica y ordenada para resolver un problema o realizar una tarea. La **estructura básica de un programa** sigue una organización común, independientemente del lenguaje de programación utilizado.

Esta estructura facilita la **lectura, mantenimiento y reutilización** del código, y normalmente incluye las siguientes partes:

8.1. PARTES PRINCIPALES DE LA ESTRUCTURA DE UN PROGRAMA

1. Zona de declaraciones (cabecera o importaciones)

- Aquí se incluyen las **bibliotecas, módulos o librerías necesarias** para el funcionamiento del programa.
- También pueden declararse **constantes, tipos de datos personalizados, o variables globales**.

```
// Declaración de bibliotecas o módulos
importar biblioteca
definir constantes
declarar tipos o estructuras
```

2. Definición de funciones y procedimientos

- Se definen todas las **funciones** (o métodos/procedimientos) que el programa necesita para dividir el trabajo en bloques reutilizables.
- Estas funciones pueden contener su propia lógica, variables locales y llamadas a otras funciones.

```
función nombre_funcion(parámetros)
    instrucciones
    return resultado
fin función
```

3. Función principal (punto de entrada del programa)

- Es la **parte que se ejecuta primero** cuando se lanza el programa.
- Se suele llamar main, inicio, o principal según el lenguaje.
- Desde aquí se coordinan las llamadas a funciones y se controla el flujo general del programa.

```
función principal()
    declarar variables
    llamar a funciones
    mostrar resultados
fin función
```



4. Comentarios y documentación

- Aunque no forman parte del código ejecutable, son esenciales para **explicar qué hace cada parte del programa**.
- Mejoran la legibilidad y permiten que otros (o el mismo autor en el futuro) comprendan mejor el funcionamiento del programa.

// Esto es un comentario que explica el propósito del siguiente bloque

8.2. ALGORITMOS

Un **algoritmo** es un **conjunto finito y ordenado de pasos** o instrucciones que se siguen para **resolver un problema o realizar una tarea**.

Es el núcleo fundamental de la programación: cualquier programa informático es, en el fondo, la implementación de uno o varios algoritmos.

8.2.1. Características de un algoritmo

Para que un conjunto de pasos se considere un algoritmo, debe cumplir con las siguientes propiedades:

- **Finitud:** Debe terminar después de un número limitado de pasos.
- **Precisión:** Cada paso debe estar claramente definido y sin ambigüedades.
- **Orden:** Las instrucciones deben seguir una secuencia lógica.
- **Eficiencia:** Debe alcanzar el objetivo utilizando los recursos de forma razonable.
- **Entrada y salida:** Un algoritmo puede recibir datos (entrada) y produce resultados (salida).

8.2.2. Representación de un algoritmo

Un algoritmo puede representarse de varias formas:

- **Lenguaje natural estructurado** (pseudocódigo).
- **Diagramas de flujo**.
- **Código en un lenguaje de programación**.

8.2.3. Tipos de algoritmos según su propósito

- **Algoritmos de búsqueda:** localizan un valor en una estructura de datos (ej. búsqueda secuencial, binaria).
- **Algoritmos de ordenación:** organizan datos (ej. burbuja, selección, quick sort).
- **Algoritmos recursivos:** se llaman a sí mismos para resolver subproblemas.
- **Algoritmos de control de flujo:** permiten tomar decisiones o repetir acciones (if, while, for).
- **Algoritmos de cifrado o compresión:** transforman datos para seguridad o ahorro de espacio.



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución, comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.